# word 2 vec

goal: To convert text to number vector , this is also called as Embeddings

1. Bag of Words (BoW)
2. Tf − idf : Term frequency * Inverse document frequency

Draw backs

- both are traditional appraoach simple math way
- both are not able to find the semantic meaning between words from a given senetence
- also using above methods the vectors size is sparse, so much computaion required

word2vec

- it is a pretrained model developed by google in 2013
- first time the embedding generating using a Neural network approach
- google trained on huge corpora around 100 billion words data
- in that they pick 3M words as vocabulary
- it is input layer(look up table) − one hidden layer(no activation − No bias) − one output layer
- the number of neurons has 300 or 100 in hidden layer
- it has two approaches
- CBOW: continoues bag of words
    - predicting the target word from surronding words
- skip gram
    - predicting the surrounding words using target word

**CBOW: continoues bag of words**

example : consider a sentence
[the cat sat on mat,
the mat sat on cat]

**step − 1: we need to create the vocabulary**

- google already created a vocabulary: 3M
- but imagine you are working only on above two sentences
- vocabulary size = 5 {cat, mat, on, sat, the}

$step-2: set\ up\ your\ target\ word$

*the cat sat on the mat:* *target word: sat*

*we want to know semantic meaning with surrounding words*

$step-3: n-gram$:

| Window Size | Text | Skip-grams |
|---|---|---|
| 2 | [ The **wide** road shimmered ] in the hot sun. | wide, the<br>wide, road<br>wide, shimmered |
| | The [ wide road **shimmered** in the ] hot sun. | shimmered, wide<br>shimmered, road<br>shimmered, in<br>shimmered, the |
| | The wide road shimmered in [ the hot **sun** ]. | sun, the<br>sun, hot |
| 3 | [ The **wide** road shimmered in ] the hot sun. | wide, the<br>wide, road<br>wide, shimmered<br>wide, in |
| | [ The wide road **shimmered** in the hot ] sun. | shimmered, the<br>shimmered, wide<br>shimmered, road<br>shimmered, in<br>shimmered, the<br>shimmered, hot |
| | The wide road shimmered [ in the hot **sun** ]. | sun, in<br>sun, the<br>sun, hot |

- *instead of taking all the surrounding words*
- *we can limit the number of words i.e. ngram*
- *assume that we are using bi gram (assume stop words are not implemeted)*
  *but in realtime will apply stop words*
  - *sat left side we have two words: the, cat*

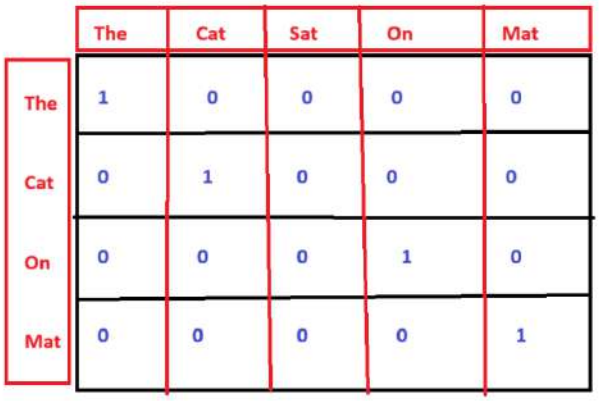
- *sat right side we have two words: on, mat*

- *assume that we are using uni gram*
  - *sat left side we have one words: cat*
  - *sat right side we have one words: on*

*step* − 4: *create one hot encoder for the input words and target word*

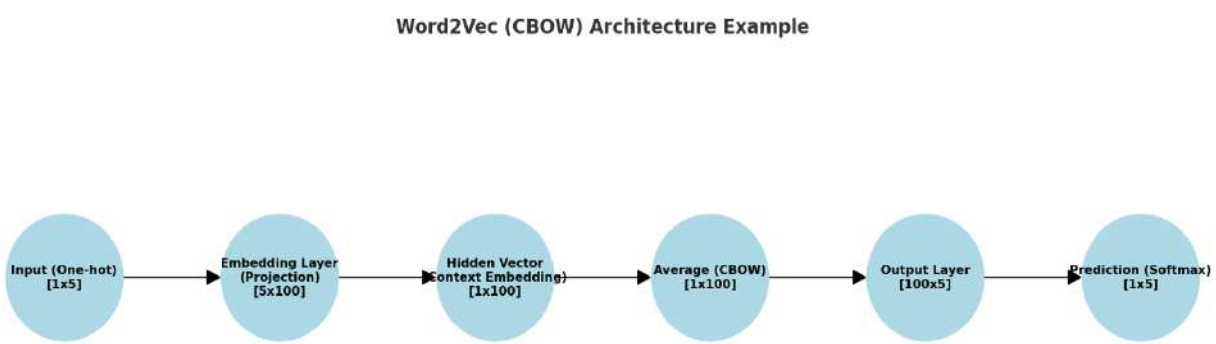
- *assume that we are using bi gram*
  - *input*: {*the*, *cat*, *on*, *mat*}
  - *output*: {*sat*}

*vocabulary size* = 5 {*cat*, *mat*, *on*, *sat*, *the*}




| | *cat* | *mat* | *on* | *sat* | *the* |
|---|---|---|---|---|---|
| *cat* | 1 | 0 | 0 | 0 | 0 |
| *on* | 0 | 0 | 1 | 0 | 0 |
| *sat* | 0 | 0 | 0 | 1 | 0 |

$$cat = \{1,0,0,0,0\}, on = \{0,0,1,0,0\}, sat = \{0,0,0,1,0\}$$



# ***Embedding layer***:

cat → [1,0,0,0,0]

on → [0,1,0,0,0]

the → [0,0,1,0,0]

mat → [0,0,0,1,0]

sat → [0,0,0,0,1]

- In a **classic feedforward neural net**, the first weight matrix projects the **input layer** into a **hidden layer**.
- In **Word2Vec**, this first projection is the **embedding lookup**.
- So the **embedding layer = hidden layer (without activation)**.

$$[1 * 5] * [5 * 100] = [1 * 100]$$

| | The | Cat | On | The | Mat | Embedding matrix | $Embedding\ vector = 1 * 100$ |
|---|---|---|---|---|---|---|---|
| $v_{the}$ | 1 | 0 | 0 | 0 | 0 | $[w_{11}, w_{12}, w_{13} \ldots . w_{1100}]$ | $E_{the} = 1X100$ |
| $v_{cat}$ | 0 | 1 | 0 | 0 | 0 | $[w_{21}, w_{22}, w_{23} \ldots . w_{2100}]$ | $E_{cat} = 1X100$ |
| $v_{on}$ | 0 | 0 | 1 | 0 | 0 | $[w_{31}, w_{32}, w_{33} \ldots . w_{3100}]$ | $E_{on} = 1X100$ |
| $v_{the}$ | 0 | 0 | 0 | 1 | 0 | $[w_{41}, w_{42}, w_{43} \ldots . w_{5100}]$ | $E_{the} = 1X100$ |
| $v_{mat}$ | 0 | 0 | 0 | 0 | 1 | $[w_{51}, w_{52}, w_{53} \ldots . w_{5100}]$ | $E_{Mat} = 1X100$ |

$$[X]_{1X5} \cdot [W]_{5X100} = [E]$$

***Average Embedding***

$$Sentence\ Embedding\ vectors = (E_{the} + \ E_{cat} + E_{on} + E_{mat})\frac{}{4}$$

$$Embedding\ layer\ or\ Hidden\ layer\ will\ produce\ 100\ values\ of\ a\ vector$$

$$\boldsymbol{Second\ embedding\ Matrix = W'}$$

| Hidden dim | $col_1$ (word "the") | $col_2$ (word "cat") | $col_3$ (word "on") | $col_4$ (word "mat") | $col_5$ (word "sat") |
|---|---|---|---|---|---|
| $h_1$ | $w'_{11}$ | $w'_{12}$ | $w'_{13}$ | $w'_{14}$ | $w'_{15}$ |
| $h_2$ | $w'_{21}$ | $w'_{22}$ | $w'_{23}$ | $w'_{24}$ | $w'_{25}$ |
| ... | ... | ... | ... | ... | ... |
| $h_{100}$ | $w'_{1001}$ | $w'_{1002}$ | $w'_{1003}$ | $w'_{1004}$ | $w'_{1005}$ |

**Multiplication**

Hidden vector = [1×100]
Output matrix = [100×5]

$$o = H * W' = [1 * 100] * [100 * 5] = [1 * 5]$$

$$These\ 5\ scores\ are\ logits\ \ one\ per\ one\ word$$

$$After\ softmax:$$

$$P\left(\frac{w}{context}\right) = \frac{e^{z_j}}{\sum_{j=1}^{n} e^{z_j}}$$

$$The\ above\ NN\ is\ CBOW\ method$$

$$1.\ Input\ layer: needs\ Contextual\ words\ vectors$$

$$2.\ Hidden\ layer: 100{,}200\ or\ 300\ Neurons$$

$$No\ activation\ function\ just\ a\ linear\ activation$$

3. $Output\ layer: Target\ word\ vectors$

$By\ using\ NN\ method\ the\ Hidden\ layer\ weights\ will\ updated\ that\ becomes$

$embedding\ vector\ for\ target\ word$

embedding vector = [one hot ]* weights

$advantages$:

1) $the\ semantic\ relationship\ between\ words\ stored\ in\ vector\ format$
2) $The\ vectors\ size\ reduces\ to\ Number\ of\ Neurons\ in\ Hidden\ layer$
3) $Before\ the\ vector\ size\ based\ on\ vocab\ size\ its\ huge$
4) $Very\ famous\ statement\ \ King - Man + woman = queen$

$$V_{king} - V_{man} + V_{wowan} = V_{queen}$$

5. $We\ can\ able\ to\ find\ the\ semantic\ relationshop\ using\ Cosine\ similarity\ of\ two\ vectors$